

2)Develop a MapReduce Program for Word Frequency

Analysis

gedit input.txt

```
(My name is Rutuja shinde.
My name is Shreya.
I am a Hadoop student.)
```

gedit mapper.py

```
(#!/usr/bin/env python3
```

import sys

for line in sys.stdin:

```
    line = line.strip()
    words = line.lower().split()
    for word in words:
        print(f'{word}\t1"))
```

gedit reducer.py

```
(#!/usr/bin/env python3
```

import sys

word_count = {}

for line in sys.stdin:

```
    word, count = line.strip().split()
    word_count[word] = word_count.get(word, 0) + int(count)
```

for word, count in word_count.items():

```
    print(f'{word}\t{count}')
)
```

cat input.txt | python3 mapper.py | sort | python3 reducer.py

Run Using Hadoop Streaming

hdfs dfs -mkdir /input

hdfs dfs -put input.txt /input

hadoop jar /home/admin1/hadoop-3.3.1/share/hadoop/tools/lib/hadoop-streaming-3.3.1.jar \

-file mapper.py -mapper mapper.py \

-file reducer.py -reducer reducer.py \

-input /input/input.txt -output /output

View Output in HDFS

hdfs dfs -ls /output

hdfs dfs -cat /output/part-00000

3)Implement Matrix Multiplication Using MapReduce

gedit input.txt

```
(A 1 1 1
A 1 2 2
A 2 1 3
A 2 2 4
B 1 1 5
B 1 2 6
B 2 1 7
B 2 2 8
)
```

gedit mapper.py

```
(#!/usr/bin/env python3
import sys
```

```
# Define matrix dimensions (adjust as per your input)
M = 2 # Rows in A
N = 2 # Columns in A / Rows in B
P = 2 # Columns in B
```

```
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    matrix, i, j, value = line.split()
    i, j, value = int(i), int(j), int(value)
```

```
    if matrix == 'A':
        # A[i][j] affects all results C[i][k] for k = 1..P
        for k in range(1, P + 1):
            print(f'{i},{k}\tA,{j},{value}')
    else: # matrix == 'B'
        # B[j][k] affects all results C[i][k] for i = 1..M
        for i_b in range(1, M + 1):
            print(f'{i_b},{j}\tB,{i},{value}')
)
```

```
gedit reducer.py
(#!/usr/bin/env python3
import sys
from collections import defaultdict
```

```
current_key = None
A_values = defaultdict(int)
B_values = defaultdict(int)
```

```
def emit_result(i, k, total):
    print(f'{i},{k}\t{total}')
```

```
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    key, value = line.split('\t')
    i, k = map(int, key.split(','))
    matrix, j, val = value.split(',')
    j, val = int(j), int(val)
```

```
    if current_key != (i, k):
        # Compute result for previous
        key
        if current_key is not None:
            total = 0
            for x in range(1, 3): # j = 1..N
                total += A_values[x] *
                B_values[x]
            emit_result(current_key[0],
            current_key[1], total)
        # Reset for new key
        current_key = (i, k)
        A_values.clear()
        B_values.clear()
```

```
    # Store partial values
    if matrix == 'A':
        A_values[j] = val
    else:
        B_values[j] = val
```

```
# Final key emission
if current_key is not None:
    total = 0
    for x in range(1, 3):
        total += A_values[x] * B_values[x]
    emit_result(current_key[0],
    current_key[1], total)
)
```

cat input.txt | python3 mapper.py |
sort | python3 reducer.py

```
9)hive
# Step 1 — Start Hadoop and Hive
$HADOOP_HOME/sbin/start-dfs.sh
$HADOOP_HOME/sbin/start-yarn.sh
hive
# Step 2 — Prepare Local Data Files
mkdir -p ~/hive_data
cat > ~/hive_data/products.csv << EOF
1,Mobile,Electronics,15000
2,Laptop,Electronics,55000
3,Shirt,Clothing,1200
4,Shoes,Clothing,2500
EOF
cat > ~/hive_data/products_electronics.csv << EOF
1,Mobile,Electronics,15000
2,Laptop,Electronics,55000
EOF
cat > ~/hive_data/products_clothing.csv << EOF
3,Shirt,Clothing,1200
4,Shoes,Clothing,2500
EOF
# Step 3 — Enter Hive Shell
hive
CREATE DATABASE IF NOT EXISTS productdb;
USE productdb;
CREATE TABLE products(
    product_id INT,
    product_name STRING,
    category STRING,
    price FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
LOAD DATA LOCAL INPATH '/home/"${USER}"/hive_data/products.csv' INTO
TABLE products;
SELECT * FROM products;
CREATE TABLE products_partitioned(
    product_id INT,
    product_name STRING,
    price FLOAT
)
PARTITIONED BY (category STRING)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

```
exit;

hdfs dfs -mkdir -p
/user/"${USER}"/productdb/products_partitioned/category=Electronics
hdfs dfs -mkdir -p /user/"${USER}"/productdb/products_partitioned/category=Clothing


hdfs dfs -put ~/hive_data/products_electronics.csv
/user/"${USER}"/productdb/products_partitioned/category=Electronics/
hdfs dfs -put ~/hive_data/products_clothing.csv
/user/"${USER}"/productdb/products_partitioned/category=Clothing/


hive

USE productdb;


ALTER TABLE products_partitioned ADD PARTITION (category='Electronics')
LOCATION '/user/"${USER}"/productdb/products_partitioned/category=Electronics';


ALTER TABLE products_partitioned ADD PARTITION (category='Clothing')
LOCATION '/user/"${USER}"/productdb/products_partitioned/category=Clothing';


SHOW PARTITIONS products_partitioned;


SELECT * FROM products_partitioned WHERE category='Electronics';


SELECT product_name, price, price * 1.1 AS price_with_tax FROM products;


SELECT UPPER(product_name) AS upper_name, ROUND(price, 0) AS rounded_price
FROM products;


CREATE VIEW expensive_products AS
SELECT * FROM products WHERE price > 5000;


SELECT * FROM expensive_products;


CREATE INDEX idx_price ON TABLE products (price)
AS 'COMPACT'
WITH DEFERRED REBUILD;


ALTER INDEX idx_price ON products REBUILD;


INSERT OVERWRITE DIRECTORY '/user/"${USER}"/hive_output/products'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT * FROM products;


exit;

hdfs dfs -cat /user/"${USER}"/hive_output/products/*
```

4)Create a MapReduce Program to Determine Student Grades.

```
gedit input.txt

(Rutuja 85
Shreya 72
Sonal 90
Shreyash 78
Mayuresh 88
Alisha 95
Harshada 65
Arya 59
Omkar 70
Prathmesh 62
)

gedit mapper.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    try:
        name, marks = line.split()
        print(f'{name}\t{marks}')
    except ValueError:
        continue
)

gedit reducer.py
#!/usr/bin/env python3
import sys

def get_grade(marks):
    marks = int(marks)
    if marks >= 90:
        return "A+"
    elif marks >= 80:
        return "A"
    elif marks >= 70:
        return "B"
    elif marks >= 60:
        return "C"
    elif marks >= 50:
        return "D"
    else:
        return "F"

##Run on Hadoop Cluste
hdfs dfs -mkdir /grades_input
hdfs dfs -put input.txt /grades_input

hadoop jar /home/admin1/hadoop-3.3.1/share/hadoop/tools/lib/hadoop-streaming-3.3.1.jar \
-file mapper.py -mapper mapper.py \
-file reducer.py -reducer reducer.py \
-input /grades_input/input.txt -output /grades_output

##View Output
hdfs dfs -cat /grades_output/part-00000
```

```
5)Develop and Implement Applications Using Apache Spark
spark-shell --version
mkdir spark_wordcount
cd spark_wordcount
gedit input.txt
(I am Student of VPKBIET. I am Studying in AI&DS Department.)
cat input.txt
gedit spark_wordcount.py
(from pyspark import SparkContext
# Create Spark Context
sc = SparkContext("local", "WordCount")
# Read text file
text_file = sc.textFile("input.txt")
# Split lines into words, then map each word to (word, 1)
counts = text_file.flatMap(lambda line: line.split()) \
                    .map(lambda word: (word, 1)) \
                    .reduceByKey(lambda a, b: a + b)
# Save output to folder
counts.saveAsTextFile("wordcount_output")
# Stop Spark
sc.stop()
)
## RUN
spark-submit spark_wordcount.py
ls wordcount_output
cat wordcount_output/part-00000
```

```
6)Clustering
import pandas as pd
from mlxtend.frequent_patterns import apriori,
association_rules
from mlxtend.preprocessing import
TransactionEncoder

# Sample dataset
dataset = [
    ['milk', 'bread', 'eggs'],
    ['milk', 'bread'],
    ['milk', 'eggs'],
    ['bread', 'butter'],
    ['bread', 'eggs'],
    ['milk', 'bread', 'butter', 'eggs']
]

# Convert transactions into one-hot encoded
DataFrame
te = TransactionEncoder()
data = te.fit(dataset).transform(dataset)
df = pd.DataFrame(data,
columns=te.columns_)

# Generate frequent itemsets
frequent_items = apriori(df, min_support=0.3,
use_colnames=True)
print("Frequent Itemsets:")
print(frequent_items)

# Generate association rules
rules = association_rules(frequent_items,
metric="lift", min_threshold=1)
print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents',
'support', 'confidence', 'lift']])

import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load the Iris dataset from CSV file
data =
pd.read_csv("C:\\Users\\shubh\\Downloads\\Iris.csv")

# Check first few rows
print(data.head())

# Drop non-numeric columns (like Id,
Species)
X = data.drop(['Id', 'Species'], axis=1)

# Apply K-Means clustering (3 clusters for 3
species)
kmeans = KMeans(n_clusters=3,
random_state=0)
X['Cluster'] = kmeans.fit_predict(X)

# Visualize clusters (SepalLengthCm vs
SepalWidthCm)
plt.figure(figsize=(6, 4))
sns.scatterplot(x=X.iloc[:,0], y=X.iloc[:,1],
hue=X['Cluster'], palette='viridis')
plt.title("K-Means Clustering on Iris Dataset")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.show()
```

7)To perform Exploratory Data Analysis (EDA)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
df=pd.read_csv("C:\\Users\\shubh\\Downloads\\Iris.csv")
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df['Species']=le.fit_transform(df['Species'])
x=df.drop('Species',axis=1)
y=df['Species']
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), cmap="coolwarm")
plt.title("Feature Correlation Heatmap")
plt.show()
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
rf= RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(x_train, y_train)
y_pred_rf = rf.predict(x_test)
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb.fit(x_train, y_train)
y_pred_gb = gb.predict(x_test)
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

```
models = {
    'Random Forest': (y_pred_rf, rf),

    'Gradient Boosting': (y_pred_gb, gb)
}
for name, (y_pred, model) in models.items():
    print(f"\n 🍀 {name} Results:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f'{name} - Confusion Matrix')
    plt.show()
```

```
from sklearn.model_selection import cross_val_score
for name, (y_pred, model) in models.items():
    cv_scores = cross_val_score(model, x, y, cv=5)
    print(f'{name} Cross-validation Accuracy: {np.mean(cv_scores):.4f}')
```

8)Analyze Titanic Disaster Data Using MapReduce

```
## folder create
mkdir -p ~/titanic_mapreduce
cd ~/titanic_mapreduce
याचा अर्थ, तुम्ही /home/admin1/titanic_mapreduce नावाच्या
फोल्डरमध्ये काम करत आहात.
आता फक्त तुमची titanic.csv फाइल त्या फोल्डरमध्ये ठेवा.
head -n 5 titanic.csv
gedit mapper_avg_age_male.py
(import sys, csv
reader = csv.DictReader(sys.stdin)
for row in reader:
    try:
        survived = row.get('Survived', '').strip()
        sex = row.get('Sex', '').strip().lower()
        age = row.get('Age', '').strip()
    except Exception:
        continue
```

```
        if survived == '0' and sex == 'male' and age not in ('', 'NA', 'None'):
            print(f"male\t{age}")
chmod +x mapper_avg_age_male.py
gedit reducer_avg_age_male.py
#!/usr/bin/env python3
import sys
total_age = 0.0
count = 0
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parts = line.split('\t')
    if len(parts) != 2:
        continue
    key, val = parts
    try:
        age = float(val)
    except:
        continue
    total_age += age
    count += 1
```

```
if count > 0:
    avg = total_age / count
    print(f"average_age_deceased_males\t{avg:.2f}")
else:
    print("average_age_deceased_males\tNA")
```

```
chmod +x reducer_avg_age_male.py
```

```
cat titanic.csv | python3 mapper_avg_age_male.py | sort | python3
reducer_avg_age_male.py
gedit mapper_female_class.py
#!/usr/bin/env python3
import sys, csv
reader = csv.DictReader(sys.stdin)
for row in reader:
    try:
        survived = row.get('Survived', '').strip()
        sex = row.get('Sex', '').strip().lower()
        pclass = row.get('Pclass', '').strip()
    except Exception:
        continue
```

```
        if survived == '0' and sex == 'female' and pclass:
            print(f"{pclass}\t1")
chmod +x mapper_female_class.py
gedit reducer_female_class.py
#!/usr/bin/env python3
```

```
import sys
counts = {}
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parts = line.split('\t')
    if len(parts) != 2:
        continue
    cls, val = parts
    try:
        counts[cls] = counts.get(cls, 0) + int(val)
    except:
        continue
for cls in sorted(counts.keys(), key=lambda x: int(x)):
    print(f"class_{cls}\t{counts[cls]}")
chmod +x reducer_female_class.py
cat titanic.csv | python3 mapper_female_class.py | sort | python3 reducer_female_class.py
## Hadoop वर रन करण्यासाठी
hdfs dfs -mkdir -p /titanic_input
hdfs dfs -put -f titanic.csv /titanic_input/
hdfs dfs -rm -r -f /titanic_output_males
hadoop jar /home/admin1/hadoop-3.3.1/share/hadoop/tools/lib/hadoop-streaming-3.3.1.jar \
-file mapper_avg_age_male.py -mapper mapper_avg_age_male.py \
-file reducer_avg_age_male.py -reducer reducer_avg_age_male.py \
-input /titanic_input/titanic.csv -output /titanic_output_males
```

```
hdfs dfs -cat /titanic_output_males/part-00000
```

```
hdfs dfs -rm -r -f /titanic_output_female_class
```

```
hadoop jar /home/admin1/hadoop-3.3.1/share/hadoop/tools/lib/hadoop-streaming-3.3.1.jar \
-file mapper_female_class.py -mapper mapper_female_class.py \
-file reducer_female_class.py -reducer reducer_female_class.py \
-input /titanic_input/titanic.csv -output /titanic_output_female_class
```